Instagram Follower Tracker — Documentação Técnica Completa

1. Visão Geral do Projeto

1.1 Propósito

Sistema de tracking de seguidores Instagram baseado em exportações manuais de dados da Meta. Processa ZIPs exportados pelo utilizador, compara snapshots consecutivos, mantém histórico completo, e gera dashboard HTML auto-contido com visualizações e métricas.

1.2 Abordagem Técnica

- **Zero dependências externas** além de Python stdlib e Chart.js (via CDN)
- **Seguro**: usa dados exportados oficialmente pela Meta, não viola ToS do Instagram
- **Offline-first**: todo o processamento é local
- **Single HTML output**: dashboard gerado é auto-contido (dados embedidos via JSON)
-

1.3 Fluxo de Utilização Semanal

```
1. Utilizador pede export no Instagram (Settings → Download your information)
2. Meta envia ZIP por email (formato JSON)
3. Utilizador coloca ZIP em snapshots/input/
4. Corre: python main.py
5. Script processa, gera dashboard.html, abre browser automaticamente
6. ZIP movido para snapshots/processed/
```

---

2. Arquitetura e Estrutura de Ficheiros

2.1 Estrutura de Diretórios

```
instagram-tracker/
├── main.py                      # Entry point root
├── dashboard.html               # Output gerado (auto-contido com dados)
│
├── src/                         # Core Python package
│   ├── __init__.py             # Package marker
│   ├── config.py               # Constantes e paths
│   ├── parser.py               # ZIP/JSON parsing
│   ├── processor.py            # Snapshot comparison logic
│   ├── storage.py              # Load/save operations
│   ├── generator.py            # HTML generation
│   └── tracker.py              # Main orchestrator
│
├── assets/                      # Frontend template e assets
│   ├── dashboard.html          # Template HTML (com __DATA_PLACEHOLDER__)
│   ├── dashboard.js            # Frontend logic
│   └── style.css               # Estilos
│
├── snapshots/                   # Data persistence
│   ├── input/                  # ZIPs novos (processados → moved to processed/)
│   ├── processed/              # ZIPs já processados (arquivo)
│   ├── history.json            # Histórico de snapshots (para dashboard)
│   └── master_followers.json   # Estado de todos os followers (dict)
│
└── README.md                    # Documentação utilizador
```

2.2 Responsabilidades dos Módulos

Módulo	Responsabilidade	Input	Output
`main.py`	Entry point, importa tracker.main()	—	—
`config.py`	Define paths, constantes, encoding	—	Constantes
`parser.py`	Extrai usernames de ZIPs	Path do ZIP	Set de usernames
`processor.py`	Compara snapshots, calcula gained/lost	ZIPs novos	History atualizado
`storage.py`	I/O de JSONs (history, followers)	—	Dados carregados/salvos
`generator.py`	Injeta dados no template HTML	History + followers dict	HTML string
`tracker.py`	Orquestra pipeline completo	—	dashboard.html + browser

---

3. Modelos de Dados (JSON Schemas)

3.1 `snapshots/history.json`

Array de snapshots processados. Cada entry representa um export ZIP.

```
[
  {
    "date": "2025-04-10",           // YYYY-MM-DD (extraído do filename ou
mtime)
    "total": 2847,                   // Total followers neste snapshot
    "gained": ["user1", "user2"],    // Usernames novos vs snapshot anterior
    "lost": ["user3"],               // Usernames que desapareceram vs anterior
    "followers": ["user1", "user2", "user4", ...] // Lista completa (sorted)
  },
  {
    "date": "2025-04-17",
    "total": 2850,
    "gained": ["user5", "user6", "user7"],
    "lost": [],
    "followers": ["user1", "user2", "user4", "user5", ...]
  }
]
```

Nota crítica: O primeiro snapshot tem `gained` e `lost` vazios (baseline). Diferenças só aparecem a partir do segundo.

3.2 `snapshots/master_followers.json`

Dicionário com estado de cada follower. Chave = username.

```
{
  "fgphoto_": {
    "first_seen": "2025-03-15",   // Data do primeiro snapshot onde apareceu
    "last_seen": "2025-04-17",    // Data do último snapshot onde apareceu
    "status": "active"             // "active" | "lost"
  },
  "another_user": {
    "first_seen": "2025-04-10",
    "last_seen": "2025-04-10",
    "status": "lost"               // Apareceu num snapshot mas desapareceu
depois
  }
}
```

Lógica de status:

- `active`: presente no snapshot mais recente
- `lost`: já não está presente no snapshot mais recente (unfollowed)

Uso:

- Tooltips no frontend (first_seen, last_seen)
- Métricas "since tracking started" (total gained ever, total lost ever)
- Futura feature de loyalty metrics (tempo médio de permanência)

3.3 Estrutura do ZIP da Meta

```
instagram-user-2025-04-17-xyz.zip
└── connections/
    └── followers_and_following/
        ├── followers_1.json       # Primeiro batch (se muitos followers)
        ├── followers_2.json       # Segundo batch (opcional)
        └── following.json         # Ignorado pelo script
```

3.4 Estrutura Interna do `followers_*.json`

```
[
  {
    "string_list_data": [
      {
        "href": "https://www.instagram.com/username1",
        "value": "username1",
        "timestamp": 1648291200
      }
    ]
  },
  {
    "string_list_data": [
      {
        "href": "https://www.instagram.com/username2",
        "value": "username2",
        "timestamp": 1648377600
      }
    ]
  }
]
```

O que extraímos: `entry["string_list_data"][i]["value"]` para cada entry.

---

4. Pipeline Python — Deep Dive

4.1 Entry Point: `main.py`

```
#!/usr/bin/env python3
from src.tracker import main

if __name__ == "__main__":
    main()
```

Simples. Delega tudo para `src.tracker.main()`.

4.2 `src/config.py`

Define constantes usadas por todos os módulos.

```
from pathlib import Path

INPUT_DIR = Path("snapshots/input")
PROCESSED_DIR = Path("snapshots/processed")
ASSETS_DIR = Path("assets")

HISTORY_FILE = Path("snapshots/history.json")
FOLLOWERS_FILE = Path("snapshots/master_followers.json")
OUTPUT_HTML = Path("dashboard.html").resolve()
TEMPLATE_FILE = ASSETS_DIR / "dashboard.html"

ENCODING = "utf-8"
```

Nota: `OUTPUT_HTML` usa `.resolve()` para obter path absoluto — garante que `webbrowser.open()` funciona em qualquer working directory.

4.3 `src/parser.py`

Responsável por extrair usernames de ZIPs da Meta.

Função: `extract_usernames(zip_path: Path) -> set`

Lógica:

1. Abre ZIP sem extrair (usa `zipfile.ZipFile`)
2. Procura ficheiros com estas condições:
   - Path contém `connections/followers_and_following`
   - Filename contém "followers"
   - Filename NÃO contém "following" (evita `following.json`)
   - Extensão é `.json`
3. Para cada ficheiro encontrado:
   - Carrega JSON
   - Se é lista → usa directamente
   - Se é dict → procura key `relationships_followers`, `followers`, ou `data`
   - Itera sobre entries
   - Para cada entry, extrai `string_list_data` → `value`
4. Retorna set de usernames (duplicados eliminados automaticamente)

Edge cases tratados:

- ZIPs com múltiplos `followers_N.json` (contas com muitos followers)
- Estrutura JSON diferente (tenta várias keys conhecidas)
- Ficheiros corrompidos (try/except com skip)

Exemplo de uso:

```
usernames = extract_usernames(Path("instagram-2025-04-17.zip"))
# → {'user1', 'user2', 'user3', ...}
```

Função: `get_snapshot_date(zip_path: Path) -> str`

Lógica:

1. Procura padrão `YYYY-MM-DD` ou `YYYY_MM_DD` no filename
2. Se encontra → retorna formatado como `YYYY-MM-DD`
3. Se não encontra → usa file modification time

Exemplo:

```
get_snapshot_date(Path("instagram-fgphoto-2025-04-17-abc123.zip"))
# → "2025-04-17"

get_snapshot_date(Path("my-export.zip"))  # sem data no nome
# → "2025-04-20"  (mtime do ficheiro)
```

4.4 `src/storage.py`

Abstração para I/O de ficheiros JSON.

Funções:

`load_history() -> list`

- Lê `snapshots/history.json`
- Retorna lista de snapshots
- Se não existe ou não é lista → retorna `[]`

`save_history(history: list)`

- Cria pasta `snapshots/` se não existir
- Escreve JSON formatado (indent=2, ensure_ascii=False)

`load_followers_dict() -> dict`

- Lê `snapshots/master_followers.json`
- Retorna dicionário `{username: {first_seen, last_seen, status}}`
- Se não existe ou não é dict → retorna `{}`

`save_followers_dict(followers: dict)`

- Escreve dicionário de followers em JSON formatado

Encoding: Todas as operações usam UTF-8 para suportar usernames com caracteres não-ASCII.

4.5 `src/processor.py`

Core logic do tracking. Compara snapshots e actualiza estado.

Função: `process_snapshots() -> list`

Fluxo completo:

```
1. Carrega history.json e master_followers.json existentes
2. Cria set de datas já processadas (para evitar duplicados)
3. Itera sobre todos os .zip em snapshots/input/ (sorted)
4. Para cada ZIP:

   a) Extrai data do filename
   b) Se data já processada → skip

   c) Extrai usernames do ZIP (via parser.extract_usernames)

   d) Calcula "active_prev" = todos os users com status != "lost" no dict

   e) Compara:
      - gained = current_snap - active_prev
      - lost = active_prev - current_snap

   f) Actualiza master_followers.json:
      - Para cada user em current_snap:
        • Se novo → cria entry {first_seen: date, last_seen: date, status:
active}
        • Se já existe → actualiza last_seen e muda status para active
      - Para cada user em lost:
        • Muda status para "lost"

   g) Adiciona entry a history:
      {date, total, gained, lost, followers}

   h) Move ZIP para snapshots/processed/

5. Salva history.json e master_followers.json actualizados
6. Retorna history completo
```

Exemplo de execução:

```
Snapshot 1 (2025-04-10): [userA, userB, userC]
→ Baseline. gained=[], lost=[]

Snapshot 2 (2025-04-17): [userA, userB, userD]
→ gained=[userD], lost=[userC]
→ master_followers atualizado:
  - userA: {first_seen: 2025-04-10, last_seen: 2025-04-17, status: active}
  - userB: {first_seen: 2025-04-10, last_seen: 2025-04-17, status: active}
  - userC: {first_seen: 2025-04-10, last_seen: 2025-04-10, status: lost}
  - userD: {first_seen: 2025-04-17, last_seen: 2025-04-17, status: active}
```

Output no terminal:

```
  [processing] instagram-2025-04-17.zip -> 2025-04-17
     📦 Reading connections/followers_and_following/followers_1.json
  ✔️ Extracted 2847 usernames from followers_1.json
     [done] Moved to snapshots/processed/

  [done] History updated (2 snapshots)
  [done] Master followers updated (2844 active)
```

4.6 `src/generator.py`

Gera dashboard HTML final injetando dados no template.

Função: `generate_html(history: list, total_active: int, followers_dict: dict) -> str`

Lógica:

1. Lê template de `assets/dashboard.html`
2. Cria objeto JSON:

   ```
   {  "snapshots": history,  "totalActive": total_active,  "followers": followers_dict}
   ```

3. Serializa para JSON string (indent=2, ensure_ascii=False)
4. Substitui `__DATA_PLACEHOLDER__` no template pelo JSON
5. Retorna HTML completo como string

Resultado: HTML auto-contido. Dados embedidos inline em `<script>const DATA = {...}</script>`.

4.7 `src/tracker.py`

Orquestrador principal.

Função: `main()`

```
def main():
    # Handle Windows console encoding (evita crashes com UTF-8)
    if sys.platform == 'win32':
        sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8')

    print("\n> Starting Instagram Follower Tracker\n")

    # 1. Processa novos snapshots
    history = process_snapshots()

    # 2. Carrega estado de followers
    followers_dict = load_followers_dict()

    # 3. Calcula total de activos
    total_active = len([u for u, i in followers_dict.items()
                        if i["status"] == "active"])

    # 4. Gera HTML
    html = generate_html(history, total_active, followers_dict)
    OUTPUT_HTML.write_text(html, encoding="utf-8")

    print(f"\n  OK Dashboard saved -> {OUTPUT_HTML}")

    # 5. Abre browser
    webbrowser.open(OUTPUT_HTML.as_uri())
```

Windows encoding fix: Necessário porque usernames podem ter emojis ou caracteres Unicode. Sem este fix, o print() crasharia no Windows.

5. Frontend — Deep Dive

5.1 Template HTML: `assets/dashboard.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>fgphoto — Follower Tracker</title>

  <!-- Fonts -->
  <link href="https://fonts.googleapis.com/css2?
family=Bebas+Neue&family=Barlow:wght@300;400;500;600&family=Barlow+Condensed:wgh
t@400;600&display=swap" rel="stylesheet">

  <!-- Chart.js -->
  <script
src="https://cdn.jsdelivr.net/npm/chart.js@4.4.0/dist/chart.umd.min.js"></
script>

  <!-- Styles -->
  <link rel="stylesheet" href="assets/style.css">
</head>
<body>
  <div class="wrapper">

    <header>
      <div class="logo">fg<em>photo</em><span
class="sep">/</span>followers</div>
      <div class="header-right">
        <div class="header-account"><strong>@fgphoto_</strong>instagram</div>
        <div class="header-updated" id="header-updated"></div>
      </div>
    </header>

    <div id="app"></div>

    <footer>
      <span>fgphoto_ · follower tracker</span>
      <span id="footer-ts"></span>
    </footer>

  </div>

  <script>
  const DATA = __DATA_PLACEHOLDER__;
  </script>
  <script src="assets/dashboard.js"></script>
</body>
</html>
```

`__DATA_PLACEHOLDER__` é substituído por:

```
const DATA = {
  "snapshots": [...],
  "totalActive": 2844,
  "followers": {...}
};
```

No output final, o dashboard.html gerado tem os dados embedidos directamente.

5.2 JavaScript: `assets/dashboard.js`

Estrutura Geral

```
// Constantes
const MONTHS = ['Jan','Feb','Mar',...];

// Funções auxiliares
function fmtDate(s) { ... }
function renderUserList(users, cls, showTooltip) { ... }
function updateDetail(idx) { ... }

// Função principal
function render() { ... }

// Execução
render();
```

Função: `fmtDate(s: string) -> string`

```
fmtDate("2025-04-17")  // → "17 Apr 2025"
fmtDate(null)          // → "—"
```

Converte YYYY-MM-DD para formato legível.

Função: `renderUserList(users: array, cls: string, showTooltip: bool) -> string`

Gera HTML para lista de usernames.

```
renderUserList(['user1', 'user2'], 'active', true)
// Retorna:
// <ul class="ulist">
//   <li><a href="https://instagram.com/user1" title="First seen: 10 Apr 2025\
nLast seen: 17 Apr 2025">@user1</a></li>
//   <li><a href="https://instagram.com/user2" title="...">@user2</a></li>
// </ul>
```

Tooltip logic:

- Se `showTooltip=true` e follower existe em `DATA.followers`:
  - Extrai `first_seen` e `last_seen`
  - Adiciona atributo `title` com texto formatado
  - Browser mostra tooltip nativo no hover

Função: `updateDetail(idx: int)`

Chamada quando o utilizador muda o selector de snapshots.

```
updateDetail(0)  // Mostra gained/lost do primeiro snapshot
```

O que faz:

1. Acede `DATA.snapshots[idx]`
2. Actualiza contadores de gained/lost
3. Re-renderiza tabelas com novos usernames

Bug corrigido aqui: Versão original usava `DATA[idx]` em vez de `DATA.snapshots[idx]`, causando erro.

Função: `render()`

Fluxo:

```
1. Validação: se DATA.snapshots vazio → mostra "no data" message e para

2. Extrai variáveis:
   - snapshots = DATA.snapshots
   - latest = último snapshot
   - prev = penúltimo snapshot (ou null)
   - activeFollowers = lista de users com status="active"
   - totalActive = count de active followers

3. Gera HTML sections:

   a) Stats "This Week":
      - Total followers (do latest snapshot)
      - Gained/Lost (latest vs prev)
      - Net change (latest vs prev)

   b) Stats "Since Tracking Started":
      - Active followers (count de active no master_followers)
      - Total gained ever (active + lost)
      - Total lost ever (count de lost)
      - Net growth (active - lost)

   c) Chart (se há 2+ snapshots):
      - Line chart com total followers ao longo do tempo

   d) Snapshot selector (se há 2+ snapshots):
      - Dropdown para escolher snapshot

   e) Gained/Lost tables:
      - Tabelas side-by-side com new followers e unfollowed

   f) Active followers section:
      - Search bar
      - Lista completa de followers activos em grid 3 colunas

   g) History table:
      - Tabela com todos os snapshots (reversed, newest first)
      - Colunas: Date, Total, Gained, Lost, Net

4. Injeta HTML no DOM: document.getElementById('app').innerHTML = ...

5. Inicializa Chart.js (se aplicável)

6. Event listeners:
   - Snapshot selector → updateDetail()
   - Search input → filtragem em tempo real
```

Search Filter Logic

```
searchInput.addEventListener('input', function() {
  const query = this.value.toLowerCase().trim();
  const items = followersList.querySelectorAll('li');
  let visibleCount = 0;

  items.forEach(item => {
    const username = item.textContent.toLowerCase();
    if (username.includes(query)) {
      item.style.display = 'flex';
      visibleCount++;
    } else {
      item.style.display = 'none';
    }
```

```
  });

  // Update counter
  searchCount.textContent = `${visibleCount} of ${activeFollowers.length}`;

  // Red counter if no results
  if (visibleCount === 0 && query.length > 0) {
    searchCount.style.color = 'var(--loss)';
  } else {
    searchCount.style.color = 'var(--muted)';
  }
});
```

Performance: Com 2800 followers, a filtragem é instantânea. O browser apenas esconde/mostra DOM nodes existentes (não re-renderiza).

5.3 CSS: `assets/style.css`

Design System

```
:root {
  --bg:         #0a0a0a;  /* Background principal */
  --bg2:        #111111;  /* Cards */
  --bg3:        #1a1a1a;  /* Inputs, hover states */
  --border:     #1f1f1f;  /* Borders */
  --text:       #f0ece4;  /* Texto principal */
  --muted:      #555;     /* Texto secundário */
  --accent:     #e8a020;  /* Accent (laranja/dourado) */
  --gain:       #4ade80;  /* Verde (gained) */
  --loss:       #f87171;  /* Vermelho (lost) */
}
```

Fonts:

- `Bebas Neue` — números grandes (stats, logo)
- `Barlow` — corpo de texto
- `Barlow Condensed` — labels uppercase

Key Components

Stats Grid:

```
.stats {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  border: 1px solid var(--border);
  gap: 1px;  /* Cria "border" visual entre cards */
}
```

User Lists (gained/lost/active):

```
.ulist {
  display: grid;
  grid-template-columns: 1fr 1fr;  /* 2 cols para gained/lost */
  /* ou */
  grid-template-columns: repeat(3, 1fr);  /* 3 cols para active followers */
}
```

Search Bar:

```
.search-bar input {
  flex: 1;
```

```
  background: var(--bg3);
  border: 1px solid var(--border);
}
.search-bar input:focus {
  border-color: var(--accent);  /* Highlight ao focar */
}
```

Animations

```
@keyframes fadeUp {
  from { opacity: 0; transform: translateY(14px); }
  to   { opacity: 1; transform: translateY(0); }
}

header { animation: fadeUp 0.35s ease both; }
.stats { animation: fadeUp 0.35s 0.05s ease both; }
/* Stagger delays para efeito cascata */
```

Responsive Breakpoint

```
@media (max-width: 780px) {
  .stats { grid-template-columns: 1fr 1fr; }  /* 2x2 em vez de 4x1 */
  .tables { grid-template-columns: 1fr; }     /* Stack vertical */
  .followers-section .ulist { grid-template-columns: 1fr 1fr; }  /* 2 cols em
vez de 3 */
}
```

---

6. Bugs Corrigidos e Porquê

Bug 1: Selector de Snapshots Não Funcionava

Ficheiro: `assets/dashboard.js`, linha 13

Código original:

```
function updateDetail(idx) {
  const s = DATA[idx];  // ❌ ERRADO
  // ...
}
```

Problema:

- `DATA` é um objecto: `{snapshots: [...], totalActive: 2844, followers: {...}}`
- `DATA[idx]` tentava aceder index numérico num objecto → undefined
- Resultava em "Cannot read property 'gained' of undefined"

Corrigido:

```
function updateDetail(idx) {
  const s = DATA.snapshots[idx];  // ✔️ CORRECTO
  // ...
}
```

Bug 2: Conflito de Tipos em `master_followers.json`

Ficheiro: `src/config.py`

Código original:

```
FOLLOWERS_FILE = Path("snapshots/master_followers.json")
MASTER_FILE = Path("snapshots/master_followers.json")  # ❌ DUPLICADO
```

Problema:

- Duas constantes apontavam para o mesmo ficheiro
- `load_master()` esperava lista: `return set(data) if isinstance(data, list)`
- `load_followers_dict()` esperava dicionário: `return data if isinstance(data, dict)`
- Se um gravasse primeiro, o outro falhava ao ler

Causa raiz: Código legacy. `load_master()` e `save_master()` nunca são chamados no pipeline actual.

Corrigido:

- Removida constante `MASTER_FILE`
- Removidas funções `load_master()` e `save_master()` de `storage.py`

Bug 3: Profile Images Causavam Confusão

Ficheiro: `assets/dashboard.js`, `assets/style.css`

Código original:

```
const img = follower && follower.profileImage
  ? `<img src="${follower.profileImage}" ...>`
  : '';
```

Problema:

- Campo `profileImage` nunca é preenchido (Meta não exporta imagens)
- Código morto que criava expectativa falsa
- CSS tinha regras `.uimg` não usadas

Corrigido:

- Removida toda a lógica de profileImage
- Usernames aparecem apenas como `@username` clicável

---

7. Features Implementadas

Feature 1: Dual Stats Panels

"This Week" — compara último snapshot com anterior:

- Útil para tracking semanal
- Mostra variação imediata

"Since Tracking Started" — agregação total:

- Útil para ver crescimento global
- Calcula: total gained ever = active + lost
- Calcula: net growth = active - lost

Implementação:

```
// This Week
const weekNet = latest.gained.length - latest.lost.length;

// Since Tracking Started
const activeFollowers = Object.keys(followersDict)
  .filter(u => followersDict[u].status === 'active');
const lostFollowers = Object.keys(followersDict)
  .filter(u => followersDict[u].status === 'lost');
const totalGainedEver = activeFollowers.length + lostFollowers.length;
const totalLostEver = lostFollowers.length;
```

Feature 2: Lista Completa de Followers Activos

Localização: Abaixo das tabelas gained/lost

Layout:

- Grid de 3 colunas (vs 2 nas outras tabelas)
- Max-height 400px com scroll
- Cada username é link clicável

Dados: Filtra `master_followers.json` onde `status === "active"`.

Feature 3: Tooltips com Histórico

Trigger: Hover sobre username na lista de active followers

Conteúdo:

```
First seen: 15 Mar 2025
Last seen: 17 Apr 2025
```

Implementação:

- Usa atributo HTML `title` (tooltip nativo do browser)
- Dados vêm de `DATA.followers[username].first_seen` e `.last_seen`

Código:

```
if (showTooltip && follower) {
  const firstSeen = follower.first_seen || '—';
```

```
  const lastSeen = follower.last_seen || '—';
  const tooltipText = `First seen: ${fmtDate(firstSeen)}\\nLast seen: $
{fmtDate(lastSeen)}`;
  tooltipAttr = ` title="${tooltipText}"`;
}
```

Feature 4: Search em Tempo Real

Localização: Topo da secção "All active followers"

Funcionalidade:

- Input field tipo texto
- Filtra lista enquanto escreve (event listener `input`)
- Case-insensitive
- Match em qualquer parte do username
- Contador dinâmico "X of Y"
- Contador fica vermelho se 0 resultados

Performance:

- Zero delay (filtragem instantânea)
- Não re-renderiza DOM (apenas `display: none`)
- Testado com 2800 followers → sem lag

Código core:

```
items.forEach(item => {
  const username = item.textContent.toLowerCase();
  if (username.includes(query)) {
    item.style.display = 'flex';
    visibleCount++;
  } else {
    item.style.display = 'none';
  }
});
```

---

8. Edge Cases e Validações

8.1 ZIP Duplicado

Cenário: Utilizador coloca mesmo ZIP duas vezes em input/

Validação: `process_snapshots()` mantém set de `existing_dates`. Se data já processada → skip com mensagem:

```
[skip] instagram-2025-04-17.zip already processed (2025-04-17), skipping
```

8.2 ZIP Sem Followers

Cenário: ZIP corrompido ou formato errado

Validação: `extract_usernames()` retorna set vazio. `process_snapshots()` detecta e skip:

```
[warn] No usernames extracted, skipping
```

ZIP não é movido para processed/ (fica em input/ para retry).

8.3 Primeiro Snapshot (Baseline)

Cenário: Histórico vazio

Comportamento:

- `gained = []`, `lost = []` (não há snapshot anterior para comparar)
- Stats cards mostram "baseline" em vez de "since YYYY-MM-DD"

8.4 Usernames com Unicode

Cenário: Usernames com emojis, cirílico, árabe, etc.

Solução:

- JSON sempre gravado com `ensure_ascii=False`
- Encoding UTF-8 em todas as operações de I/O
- Windows console fix em `tracker.py`

8.5 Múltiplos `followers_N.json` no ZIP

Cenário: Conta com muitos followers → Meta divide em batches

Solução: `extract_usernames()` itera sobre **todos** os ficheiros `followers_*.json` encontrados e agrega usernames num único set.

---

9. Dependências e Requirements

9.1 Python

- **Versão:** 3.8+
- **Stdlib apenas:** zipfile, json, re, datetime, pathlib, webbrowser, sys, io

Sem pip install necessário.

9.2 Frontend

- **Chart.js 4.4.0** — via CDN (não requer npm)
- **Google Fonts** — Bebas Neue, Barlow, Barlow Condensed

Sem build step. Zero npm.

9.3 Browser

- Moderno com suporte a ES6 (Chrome 60+, Firefox 55+, Safari 11+)

---

10. Como Testar

10.1 Teste Básico (Happy Path)

```
# 1. Coloca um ZIP em snapshots/input/
cp ~/Downloads/instagram-export.zip snapshots/input/

# 2. Corre o script
python main.py

# 3. Valida output no terminal:
#    - "Extracted X usernames"
#    - "Moved to snapshots/processed/"
#    - "Dashboard saved"

# 4. Valida ficheiros gerados:
ls snapshots/
# → history.json (deve ter 1 entry)
# → master_followers.json (deve ter X entries)

ls snapshots/processed/
# → ZIP movido para aqui

# 5. Valida dashboard.html abre no browser
# → Deve mostrar stats, chart (se 2+ snapshots), lista de followers
```

10.2 Teste de Segundo Snapshot

```
# 1. Coloca segundo ZIP (data diferente) em input/
cp ~/Downloads/instagram-export-2.zip snapshots/input/

# 2. Corre novamente
python main.py

# 3. Valida terminal:
#    - "gained=[...]" deve ter usernames
#    - "lost=[...]" pode ter usernames

# 4. Valida dashboard:
#    - Gráfico deve aparecer (2 pontos)
#    - Selector de snapshots deve aparecer
#    - Stats "This week" devem mostrar deltas correctos
```

10.3 Teste de Search

```
# 1. Abre dashboard.html
# 2. Navega para secção "All active followers"
# 3. Escreve no search field
# 4. Valida:
#    - Lista filtra enquanto escreve
#    - Contador actualiza (ex: "5 of 2844")
#    - Limpar campo → mostra todos novamente
```

10.4 Teste de Tooltips

```
# 1. Hover sobre qualquer username na lista de active followers
# 2. Valida tooltip aparece com:
#    First seen: DD MMM YYYY
#    Last seen: DD MMM YYYY
```

10.5 Teste de Selector de Snapshots

```
# (Requer 2+ snapshots)
# 1. Abre dashboard
# 2. Muda selector para snapshot mais antigo
# 3. Valida:
#    - Tabelas de gained/lost actualizam
#    - Contadores actualizam
```

---

11. Estrutura de Commit Sugerida

Se fores fazer commit para GitHub:

```
feat: initial tracker implementation
feat: add dual stats panels (this week + all time)
feat: add active followers list with 3-col grid
feat: add tooltips with first_seen/last_seen
feat: add real-time search filter
fix: snapshot selector bug (DATA[idx] → DATA.snapshots[idx])
fix: remove duplicate MASTER_FILE constant
refactor: remove unused load_master/save_master functions
docs: add comprehensive technical documentation
```

---

12. Próximos Passos Sugeridos

12.1 Loyalty Metrics

Adicionar secção "Follower Insights":

- Follower mais antigo (username com `first_seen` mais antigo)
- Média de tempo que followers ficam
- Breakdown: <1 mês, 1-3 meses, 3-6 meses, 6+ meses

Dados já disponíveis: `first_seen` no `master_followers.json`

12.2 Export para CSV

Botão "Export to CSV" na secção de active followers.

Colunas: username, first_seen, last_seen, status

Implementação: Client-side com Blob API:

```
const csv = 'username,first_seen,last_seen,status\n' +
  activeFollowers.map(u => {
    const f = DATA.followers[u];
    return `${u},${f.first_seen},${f.last_seen},${f.status}`;
  }).join('\n');

const blob = new Blob([csv], {type: 'text/csv'});
const url = URL.createObjectURL(blob);
const a = document.createElement('a');
a.href = url;
a.download = 'followers.csv';
a.click();
```

12.3 Comparação Livre de Snapshots

Expandir selector para escolher dois snapshots quaisquer (não só consecutivos).

UI: Dois dropdowns "From:" e "To:"

Logic: Calcular `gained = to.followers - from.followers`

Use case: Ver crescimento entre 10 Mar e 17 Abr (saltar snapshots intermédios)

12.4 Dark/Light Theme Toggle

Adicionar botão de toggle no header.

Implementação:

- CSS variables já estão centralizados em `:root`
- Criar segundo set de vars (light theme)
- Toggle adiciona classe `.light-theme` ao body
- CSS usa `body.light-theme { --bg: #fff; --text: #000; ... }`

12.5 Notificações de Unfollows Relevantes

Se alguém "importante" te deixar de seguir → highlight especial.

Implementação:

- Adicionar campo `important: true` em `master_followers.json` (manual)
- Secção "Lost" mostra esses users primeiro com badge especial

13. Troubleshooting Guide

Problema: "No followers JSON found"

Causa: Estrutura do ZIP mudou ou path errado

Solução:

1. Abre ZIP manualmente e verifica path exacto
2. Ajusta regex em `parser.py` se necessário

Problema: Dashboard não abre automaticamente

Causa: `webbrowser.open()` falha em alguns ambientes

Solução:

- Abre `dashboard.html` manualmente com duplo clique
- Ou: `open dashboard.html` (macOS), `start dashboard.html` (Windows)

Problema: Stats "Since tracking started" mostram 0

Causa: `master_followers.json` vazio ou corrompido

Solução:

1. Apaga `snapshots/master_followers.json`
2. Apaga `snapshots/history.json`
3. Move ZIPs de `processed/` de volta para `input/`
4. Corre `python main.py` novamente

Problema: Search não funciona

Causa: JavaScript error (provavelmente DOM não carregou)

Solução:

1. Abre browser console (F12)
2. Verifica erros JavaScript
3. Confirma que `#follower-search` existe no DOM

Problema: Chart não aparece

Causa: Chart.js não carregou do CDN

Solução:

1. Verifica ligação à internet
2. Abre browser console → Network tab
3. Confirma que `chart.js@4.4.0` carregou (status 200)

---

14. Notas Finais para Handoff

O Que Está Pronto

- Pipeline Python completo e testado
- Dashboard funcional com todas as features
- Bugs críticos corrigidos
- Documentação completa

O Que Pode Melhorar

- Testes unitários (actualmente zero coverage)
- CI/CD setup
- Packaging (setup.py para `pip install`)
- Error handling mais robusto (ex: ZIP corrompido deve mostrar mensagem clara)

Decisões Técnicas Importantes

Porquê não usar API do Instagram?

- Instagram Graph API removeu endpoint de lista de followers
- Só dá número total, não usernames
- Abordagem de export manual é mais segura e não viola ToS

Porquê HTML auto-contido em vez de SPA React?

- Zero dependencies
- Funciona offline
- Fácil de partilhar (enviar ficheiro por email)
- Não precisa de servidor

Porquê master_followers.json em vez de SQLite?

- Simplicidade
- Human-readable (fácil debug)
- Portável
- Performance é suficiente até ~10k followers

Porquê Chart.js via CDN em vez de bundle local?

- Reduz tamanho do repo
- Browser cache partilhado
- Auto-update

---

15. Contactos e Recursos

Repositório: `https://github.com/filipecosgom/Instagram-tracker`

Estrutura de suporte:

- Issues no GitHub para bugs
- README.md para documentação de utilizador
- Este documento (TECHNICAL.md) para developers

Meta Data Export:

- https://help.instagram.com/181231772500920

Chart.js Docs:

- https://www.chartjs.org/docs/latest/

Documento criado: 21 Abril 2026
Versão: 1.0
Autor: Senior Developer Handoff Documentation